

UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS,
ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE

Tema:
REDUCCIÓN MATRICIAL DE IMÁGENES MEDIANTE MUESTREO ESPACIAL, APROVECHAMIENTO DE REDUNDANCIA Y FILTRADO DISCRETO SIN FUNCIONES DE LIBRERÍA

Unidad de Organización Curricular:
Profesional

Nivel y Paralelo:
7mo – A

Asignatura:
Inteligencia Artificial

Docente:
Ing. Mg. Ruben Nogales

Alumnos participantes:
Toala Steeven
Chavez Michael

Ambato – Ecuador
Octubre de 2026

Índice

1. Introducción y Planteamiento del Problema **1**

2. Marco Teórico y Formulación Matemática **1**
2.1. Representación Matricial de la Imagen Digital 1
2.2. El Principio de Redundancia Espacial 2
2.3. Método 1: Submuestreo por Saltos (Decimación Espacial) 2
2.4. Método 2: Promedio de Bloques Redundantes (Mean Pooling) 2
2.5. Método 3: Reducción por Filtro Mediana (Median Pooling) 3
2.6. Método 4: Filtrado Gaussiano Ponderado 2D 3
2.7. Fundamentación del Uso Estricto de Matrices Impares ($3 \times 3, 5 \times 5, 7 \times 7$) 3

3. Correspondencia entre Notación Matemática y Variables del Software **4**

4. Implementación del Algoritmo en Python **4**
4.1. Código Fuente de los Algoritmos de Reducción 4
4.2. Inspección Didáctica de Matrices en Terminal y Aplicación Web 6

5. Resultados Experimentales y Discusión **6**
5.1. Evaluación Cuantitativa de Reducción y Compresión 6
5.2. Demostración Matricial del Mapeo de Píxeles 7
5.3. Análisis Visual Comparativo de los Métodos 7

6. Banco de Preguntas Técnicas para la Sustentación **8**

7. Conclusiones y Recomendaciones **9**
7.1. Conclusiones 9
7.2. Recomendaciones 9

Referencias Bibliográficas **11**

Índice de figuras

1. Curva de disminución de píxeles totales (en miles) y tasa porcentual de ahorro de datos según el factor de paso impar $k \in \{3, 5, 7\}$. 7
2. Demostración didáctica del mapeo discreto: bloque de 3×3 píxeles originales y valor numérico de salida obtenido por cada uno de los cuatro algoritmos. 7
3. Comparativa visual de los cuatro métodos de reducción para paso impar $k = 3$ (reducción a 266×200 píxeles, 88,9 % de ahorro). 8
4. Comparativa visual de los cuatro métodos de reducción para paso impar $k = 5$ (reducción a 160×120 píxeles, 96,0 % de ahorro). 8

Índice de tablas

1. Correspondencia entre la notación matemática teórica y las variables del software 4
2. Métricas cuantitativas de reducción dimensional y tasa de ahorro de datos 6

1. Introducción y Planteamiento del Problema

En el ámbito del **Procesamiento Digital de Imágenes** y la **Inteligencia Artificial**, las imágenes capturadas por sensores ópticos representan volúmenes masivos de datos digitales. En aplicaciones prácticas como la visión artificial por computador, el reconocimiento biométrico, la robótica móvil y el entrenamiento de modelos de aprendizaje profundo (*Deep Learning* mediante Redes Neuronales Convolucionales o CNNs), el procesamiento directo de imágenes en alta resolución impone una carga computacional prohibitiva en términos de memoria RAM, ancho de banda y tiempo de cómputo [1].

Frente a esta problemática, la reducción dimensional o cambio de escala espacial (*downsampling*) constituye una de las etapas esenciales de acondicionamiento de datos. No obstante, en la práctica académica y profesional contemporánea es común el empleo indiscriminado de funciones de caja negra provenientes de librerías comerciales o de código abierto, tales como `cv2.resize` en OpenCV [2] o `imresize` en MATLAB. Dichas funciones aplican de manera opaca algoritmos complejos de interpolación bilineal, bicúbica, *splines* o filtros Lanczos, ocultando los principios matemáticos fundacionales de la teoría de señales: el **muestreo espacial discreto**, la manipulación de índices en matrices bidimensionales y el aprovechamiento de la **redundancia espacial de píxeles** [3].

Por tal motivo, el presente proyecto se desarrolla bajo la restricción pedagógica explícita de **prohibición total de funciones automáticas de cambio de escala**. En su lugar, se implementan desde cero, mediante programación matricial pura en Python y NumPy [4], cuatro métodos de reducción dimensional sobre matrices impares ($3 \times 3, 5 \times 5, 7 \times 7$):

1. **Submuestreo por Decimación Espacial (Saltos):** Muestreo periódico de la matriz omitiendo píxeles intermedios.
2. **Promedio de Bloques Redundantes (*Mean Pooling*):** Integración de la luminosidad de píxeles vecinos mediante la media aritmética.
3. **Filtro Mediana de Bloques (*Median Pooling*):** Selección del elemento central ordenado con alta inmunidad a ruido impulsivo.
4. **Ponderación Gaussiana Bidimensional Discreta:** Convolución local mediante una máscara normalizada con decaimiento radial.

Los objetivos planteados para la práctica son:

- Implementar de forma manual y vectorizada los cuatro algoritmos de reducción en Python sin utilizar comandos empaquetados de redimensionamiento.
- Demostrar analítica y visualmente por qué las imágenes reducidas conservan su inteligibilidad geométrica gracias a la redundancia espacial.
- Justificar rigurosamente la necesidad matemática del empleo de matrices y saltos de orden impar ($3 \times 3, 5 \times 5, 7 \times 7$).
- Evaluar cuantitativamente la compresión de datos y la tasa de ahorro espacial obtenida para distintos factores de paso k.

2. Marco Teórico y Formulación Matemática

2.1. Representación Matricial de la Imagen Digital

Una imagen digital monocromática continua se modela matemáticamente como una función bidimensional de intensidad lumínica $f(x, y)$. En el dominio discreto computacional, tras los procesos físicos de muestreo espacial y cuantización radiométrica, la imagen se representa como una matriz

numérica bidimensional $I \in \mathbb{R}^{M \times N}$:

$$I = \begin{bmatrix} I(0,0) & I(0,1) & \cdots & I(0,N-1) \\ I(1,0) & I(1,1) & \cdots & I(1,N-1) \\ \vdots & \vdots & \ddots & \vdots \\ I(M-1,0) & I(M-1,1) & \cdots & I(M-1,N-1) \end{bmatrix} \tag{1}$$

donde cada elemento $I(i,j)$ representa un **píxel** cuya intensidad para una resolución radiométrica estándar de 8 bits está acotada en el intervalo cerrado de enteros:

$$I(i,j) \in \{0,1,2,\ldots,255\} \subset \mathbb{Z} \tag{2}$$

donde 0 denota la ausencia total de luz (negro puro) y 255 representa la saturación máxima (blanco puro). Para imágenes a color en el espacio RGB, la representación se extiende a un tensor tridimensional de orden $M \times N \times 3$, compuesto por tres matrices desacopladas correspondientes a los canales Rojo (R), Verde (G) y Azul (B) [5].

2.2. El Principio de Redundancia Espacial

En escenas del mundo físico real, los objetos presentan coherencia geométrica y superficies continuas (paredes, rostros, cielo, vestimenta). Como consecuencia física directa, dos píxeles contiguos en coordenadas (i,j) y $(i,j+1)$ exhiben una correlación estadística sumamente alta:

$$|I(i,j) - I(i,j+1)| \approx 0 \tag{3}$$

Esta propiedad se denomina **redundancia espacial**. Significa que gran parte de los píxeles adyacentes no aportan información estructural nueva, sino que replican la luminosidad de su entorno. Por ende, al reducir las dimensiones espaciales mediante muestreo o promediado, la estructura global, bordes y formas macroscópicas permanecen plenamente discernibles para el sistema visual humano.

2.3. Método 1: Submuestreo por Saltos (Decimación Espacial)

El método más elemental de reducción consiste en recorrer la matriz original dando saltos de tamaño fijo $k \in \mathbb{N}$ tanto en las filas como en las columnas. Si la matriz original posee dimensiones $M \times N$, la nueva matriz reducida I_{saltos} tendrá dimensiones:

$$M' = \left\lfloor \frac{M}{k} \right\rfloor, \quad N' = \left\lfloor \frac{N}{k} \right\rfloor \tag{4}$$

La correspondencia determinística entre las coordenadas del píxel reducido (i,j) y el píxel original viene dada por:

$$I_{\text{saltos}}(i,j) = I_{\text{original}}(i \cdot k, j \cdot k) \tag{5}$$

Análisis de Señal y Aliasing: De acuerdo con el Teorema de Muestreo de Nyquist-Shannon [6], para reconstruir o representar fidedignamente una señal continua, la frecuencia de muestreo espacial f_s debe ser al menos el doble de la frecuencia espacial máxima presente en la imagen ($f_s \geq 2f_{\text{máx}}$). Al aplicar decimación directa sin filtrado previo, las frecuencias espaciales altas que superen el límite de Nyquist se solapan sobre las bajas frecuencias, provocando artefactos de **aliasing** o escalonamiento visible (bordes dentados y patrones de Moiré).

2.4. Método 2: Promedio de Bloques Redundantes (Mean Pooling)

Para mitigar el descarte destructivo de información del submuestreo directo, el método de promedio integra la información de todos los $k \times k$ píxeles que conforman la vecindad o bloque local. Cada celda de la imagen reducida se calcula como la media aritmética de su bloque asociado:

$$I_{\text{promedio}}(i,j) = \text{redondear}\left(\frac{1}{k^2} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} I_{\text{original}}(i \cdot k + u, j \cdot k + v) \right) \tag{6}$$

Ventaja Física: Este método equivale a aplicar un filtro de caja (*Box Filter*) de paso bajo antes de la decimación. Al promediar los k^2 píxeles redundantes, se suavizan las altas frecuencias espaciales, preservando la energía lumínica total de la escena y atenuando significativamente el efecto de aliasing en los bordes.

2.5. Método 3: Reducción por Filtro Mediana (Median Pooling)

El filtro mediana es un operador no lineal de vecindad. En lugar de computar una combinación lineal ponderada, extrae el conjunto de los k^2 valores discretos del bloque:

$$\mathcal{B}_{i,j} = \{I_{\text{original}}(i \cdot k + u, j \cdot k + v) \mid 0 \leq u < k,\ 0 \leq v < k\} \tag{7}$$

Dichos valores se ordenan de menor a mayor en una secuencia monotónica $\{v_{(1)}, v_{(2)}, \ldots, v_{(k^2)}\}$. La intensidad del píxel resultante corresponde al valor ubicado en la posición central:

$$I_{\text{mediana}}(i,j) = v_{\left(\frac{k^2+1}{2}\right)} \tag{8}$$

Propiedad Antirruido: A diferencia del promedio aritmético, que es sumamente vulnerable a valores atípicos (*outliers*), la mediana es altamente robusta frente a ruido impulsivo (sal y pimienta), preservando discontinuidades y bordes afilados sin generar halos difusos.

2.6. Método 4: Filtrado Gaussiano Ponderado 2D

La convolución gaussiana asigna coeficientes de ponderación espacialmente simétricos, donde los píxeles cercanos al centro geométrico del bloque reciben mayor peso que los de la periferia, imitando la función de dispersión de punto (*Point Spread Function*) del sistema visual humano. La función gaussiana continua bidimensional isotrópica con media cero y desviación estándar σ se define como:

$$G(x,y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right) \tag{9}$$

En el espacio discreto para una matriz de tamaño $k \times k$, las coordenadas espaciales se centran en el origen mediante:

$$x, y \in \left\{-\frac{k-1}{2}, \ldots, 0, \ldots, \frac{k-1}{2}\right\}, \quad \sigma = \frac{k}{3} \tag{10}$$

Para garantizar que la transformación no altere la ganancia lumínica global de la imagen, la matriz de coeficientes $K(u,v)$ se normaliza estrictamente de forma que la suma de todos sus elementos sea idénticamente 1,0:

$$K(u,v) = \frac{G(u,v)}{\sum_p \sum_q G(p,q)}, \quad \text{tal que} \quad \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K(u,v) = 1{,}0 \tag{11}$$

El valor del píxel reducido se obtiene mediante la suma de productos discretos:

$$I_{\text{gauss}}(i,j) = \text{redondear}\left(\sum_{u=0}^{k-1} \sum_{v=0}^{k-1} I_{\text{original}}(i \cdot k + u, j \cdot k + v) \cdot K(u,v)\right) \tag{12}$$

2.7. Fundamentación del Uso Estricto de Matrices Impares ($3\times3, 5\times5, 7\times7$)

La selección obligatoria de factores de salto y matrices de orden impar obedece a tres principios matemáticos fundamentales de la visión artificial:

1. **Existencia de un Centro Geométrico Entero Simétrico:** Una matriz de orden impar $k \times k$ posee un único centro geométrico situado exactamente en las coordenadas enteras:

$$(c_x, c_y) = \left(\frac{k-1}{2}, \frac{k-1}{2}\right) \in \mathbb{N}^2 \tag{13}$$

Por el contrario, en matrices pares ($2\times2, 4\times4$), el centro teórico cae en coordenadas fraccionarias (0,5, 1,5), careciendo de un píxel físico sobre el cual anclar simétricamente la operación.

2. **Cálculo de Mediana Pura sin Aproximaciones:** Para un bloque impar, el cardinal del conjunto es $k^2 \in \{9, 25, 49\}$, el cual es siempre un número impar. En consecuencia, la mediana corresponde de forma exacta y pura a un valor real existente dentro de la muestra:

$$\text{Posición central exacta} = \frac{k^2+1}{2} \in \mathbb{Z} \tag{14}$$

En bloques pares ($k^2 = 4, 16, 64$), la mediana requeriría promediar los dos elementos centrales, desvirtuando la naturaleza no lineal del filtro.

3. **Isotropía y Simetría Radial Gaussiana:** La campana de Gauss requiere distribuirse simétricamente alrededor de un pico central máximo en $(0, 0)$. Solo las matrices impares permiten ubicar el vértice de la campana en el píxel central y distribuir los pesos equitativamente en los cuatro cuadrantes espaciales.

3. Correspondencia entre Notación Matemática y Variables del Software

Con el fin de garantizar máxima transparencia y trazabilidad entre la fundamentación matemática y el código implementado, la Tabla 1 detalla la correspondencia unívoca entre los símbolos de las ecuaciones y las variables definidas en Python.

Tabla 1: Correspondencia entre la notación matemática teórica y las variables del software

Símbolo	**Significado Conceptual**	**Variable en Python** (`main.py`)
$M \times N$	Dimensiones originales de la imagen (alto × ancho)	`alto_orig`, `ancho_orig`
$k \in \{3, 5, 7\}$	Factor de salto impar del bloque espacial	`paso_k`
$M' \times N'$	Dimensiones de la imagen reducida	`alto_nuevo`, `ancho_nuevo`
k^2	Cantidad total de píxeles por bloque $(9, 25, 49)$	`total_pixeles_bloque`
$I_{\text{original}}(i \cdot k, j \cdot k)$	Píxel discreto muestreado por saltos	`imagen_reducida[fila, col]`
$\frac{1}{k^2}\sum I$	Media aritmética del bloque redundante	`promedio`
$v_{\left(\frac{k^2+1}{2}\right)}$	Mediana del bloque ordenado	`mediana_valor`
$K(u, v)$	Kernel gaussiano bidimensional normalizado	`kernel_normalizado`
$\sigma = k/3$	Desviación estándar de la máscara gaussiana	`desviacion_estandar`
$1 - \frac{M'N'}{MN}$	Porcentaje de reducción de datos y memoria	`porcentaje_ahorro`

4. Implementación del Algoritmo en Python

El sistema fue desarrollado en el lenguaje de programación Python utilizando operaciones matriciales directas con la biblioteca `NumPy` [4] y visualización científica con `Matplotlib` [7]. Se evitó rigurosamente cualquier llamada a funciones cerradas como `cv2.resize`.

4.1. Código Fuente de los Algoritmos de Reducción

A continuación se presentan los fragmentos de código modular correspondientes a los cuatro métodos implementados en el archivo `codigo/main.py`:

```python
import numpy as np

def generar_kernel_gaussiano(paso_k: int) -> np.ndarray:
    """Genera kernel gaussiano 2D normalizado (suma total = 1.0)."""
    desviacion_estandar = paso_k / 3.0
    rango_ejes = np.linspace(-(paso_k - 1) / 2.0, (paso_k - 1) / 2.0, paso_k)
    curva_gaussiana = np.exp(-0.5 * np.square(rango_ejes) / np.square(
    desviacion_estandar))
    kernel_2d = np.outer(curva_gaussiana, curva_gaussiana)
    return kernel_2d / np.sum(kernel_2d)

def reducir_por_saltos(imagen_original: np.ndarray, paso_k: int) -> np.ndarray:
    """Muestreo espacial directo dando saltos de 'paso_k' pixeles."""
    alto_n = imagen_original.shape[0] // paso_k
    ancho_n = imagen_original.shape[1] // paso_k
    canales = imagen_original.shape[2] if imagen_original.ndim == 3 else 1

    # Decimacion espacial directa mediante saltos matriciales
    if imagen_original.ndim == 2:
        return imagen_original[::paso_k, ::paso_k][:alto_n, :ancho_n].copy()
    else:
        return imagen_original[::paso_k, ::paso_k, :][:alto_n, :ancho_n, :].copy()

def reducir_por_promedio(imagen_original: np.ndarray, paso_k: int) -> np.ndarray:
    """Aprovechamiento de redundancia mediante media aritmetica (Mean Pooling)."""
    alto_n = imagen_original.shape[0] // paso_k
    ancho_n = imagen_original.shape[1] // paso_k
    recorte = imagen_original[:alto_n * paso_k, :ancho_n * paso_k]

    if imagen_original.ndim == 2:
        bloques = recorte.reshape(alto_n, paso_k, ancho_n, paso_k)
        return np.round(bloques.mean(axis=(1, 3))).astype(np.uint8)
    else:
        canales = imagen_original.shape[2]
        bloques = recorte.reshape(alto_n, paso_k, ancho_n, paso_k, canales)
        return np.round(bloques.mean(axis=(1, 3))).astype(np.uint8)

def reducir_por_mediana(imagen_original: np.ndarray, paso_k: int) -> np.ndarray:
    """Filtro mediana sobre bloques impares con centro geometrico real."""
    alto_n = imagen_original.shape[0] // paso_k
    ancho_n = imagen_original.shape[1] // paso_k
    recorte = imagen_original[:alto_n * paso_k, :ancho_n * paso_k]

    if imagen_original.ndim == 2:
        bloques = recorte.reshape(alto_n, paso_k, ancho_n, paso_k)
        bloques_flat = bloques.transpose(0, 2, 1, 3).reshape(alto_n, ancho_n, paso_k *
     paso_k)
        return np.round(np.median(bloques_flat, axis=2)).astype(np.uint8)
    else:
        canales = imagen_original.shape[2]
        bloques = recorte.reshape(alto_n, paso_k, ancho_n, paso_k, canales)
        bloques_flat = bloques.transpose(0, 2, 4, 1, 3).reshape(alto_n, ancho_n,
    canales, paso_k * paso_k)
        return np.round(np.median(bloques_flat, axis=3)).astype(np.uint8)

def reducir_por_gaussiana(imagen_original: np.ndarray, paso_k: int) -> np.ndarray:
    """Convolucion ponderada con kernel gaussiano 2D simetrico."""
    alto_n = imagen_original.shape[0] // paso_k
    ancho_n = imagen_original.shape[1] // paso_k
    recorte = imagen_original[:alto_n * paso_k, :ancho_n * paso_k]
    kernel = generar_kernel_gaussiano(paso_k)

    if imagen_original.ndim == 2:
        bloques = recorte.reshape(alto_n, paso_k, ancho_n, paso_k)
        res = np.tensordot(bloques, kernel, axes=([1, 3], [0, 1]))
        return np.clip(np.round(res), 0, 255).astype(np.uint8)
    else:
        canales = imagen_original.shape[2]
```

```
66          bloques = recorte.reshape(alto_n, paso_k, ancho_n, paso_k, canales)
67          res = np.tensordot(bloques, kernel, axes=([1, 3], [0, 1]))
68          return np.clip(np.round(res), 0, 255).astype(np.uint8)
```

Listing 1: Implementación de los algoritmos de reducción matricial sin funciones de librería

4.2. Inspección Didáctica de Matrices en Terminal y Aplicación Web

Para complementar la comprensión de los cálculos a nivel de celda de memoria, el software cuenta con dos modalidades didácticas:

1. **Modo Terminal (`main.py`):** Imprime directamente en la consola los valores enteros reales del bloque de píxeles ubicado en la esquina superior izquierda $(0, 0)$ y muestra la traza numérica de cómo cada algoritmo computa su salida.

2. **Aplicación Web Interactiva (`app.py`):** Desarrollada en Flask con frontend reactivo en Alpine.js. Permite al usuario cargar imágenes personalizadas, alternar entre pasos impares ($3 \times 3, 5 \times 5, 7 \times 7$), seleccionar el algoritmo y editar en vivo los valores numéricos de una matriz de píxeles interactiva para observar el resultado en tiempo real.

5. Resultados Experimentales y Discusión

La validación experimental de los algoritmos implementados se llevó a cabo utilizando una imagen de prueba estándar (`imagen_ejemplo.jpg`) de dimensiones originales 800×600 píxeles a color (tres canales RGB), totalizando 480,000 píxeles por canal y 1,440,000 componentes numéricas.

5.1. Evaluación Cuantitativa de Reducción y Compresión

La Tabla 2 presenta la cuantificación rigurosa de las dimensiones espaciales, número total de píxeles resultantes y porcentaje efectivo de ahorro de datos para los tres factores de paso impar evaluados ($k = 3, 5, 7$).

Tabla 2: Métricas cuantitativas de reducción dimensional y tasa de ahorro de datos

Configuración	**Paso k**	**Dimensiones**	**Total Píxeles**	**Fracción**	**Ahorro (%)**
Original	–	800×600	480,000	1,0000	0,00 %
Paso Impar 1	$k = 3$	266×200	53,200	$\approx 1/9$	**88,92** %
Paso Impar 2	$k = 5$	160×120	19,200	$1/25$	**96,00** %
Paso Impar 3	$k = 7$	114×85	9,690	$\approx 1/49$	**97,98** %

Como se observa en los datos y se ilustra gráficamente en la Figura 1, al incrementar el factor de salto a $k = 3$, el número de píxeles se divide por un factor de 9 ($k^2 = 3^2$), liberando de forma inmediata un **88,92** % del volumen de datos en memoria. Con $k = 5$, la compresión alcanza el **96,00** %, evidenciando la potencia del muestreo espacial para optimizar el rendimiento de arquitecturas computacionales.

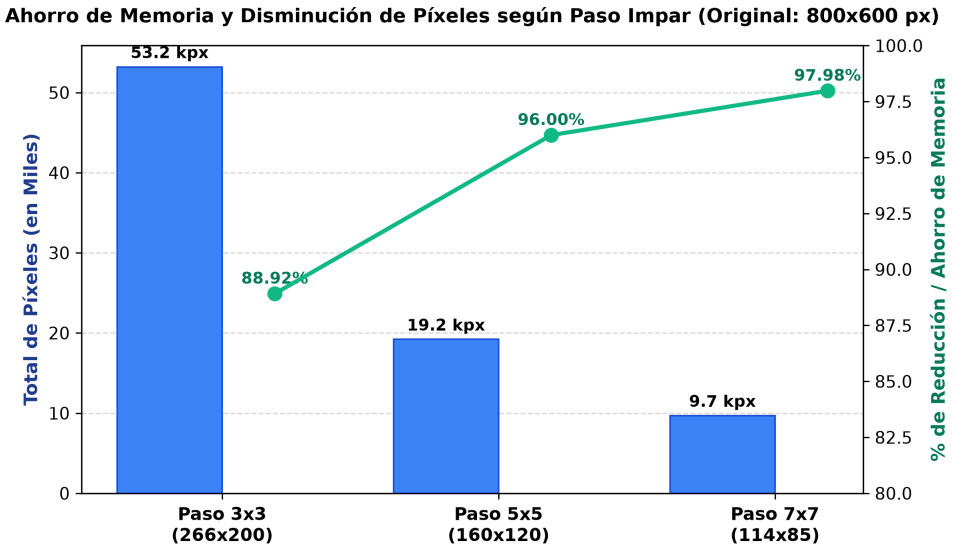


Figura 1: Curva de disminución de píxeles totales (en miles) y tasa porcentual de ahorro de datos según el factor de paso impar $k \in \{3, 5, 7\}$.

5.2. Demostración Matricial del Mapeo de Píxeles

La Figura 2 detalla el proceso físico exacto que experimenta un bloque de 3×3 píxeles contiguos en la escala de grises. Se aprecia con nitidez cómo los 9 valores de entrada son transformados determinísticamente a un único píxel escalar de salida según el operador seleccionado.

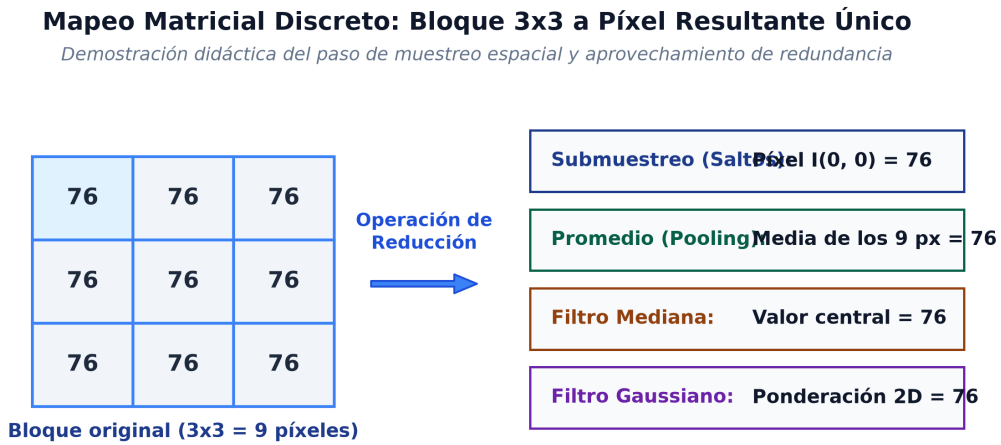


Figura 2: Demostración didáctica del mapeo discreto: bloque de 3×3 píxeles originales y valor numérico de salida obtenido por cada uno de los cuatro algoritmos.

5.3. Análisis Visual Comparativo de los Métodos

Las Figuras 3 y 4 muestran la comparativa visual de los resultados obtenidos para pasos $k = 3$ y $k = 5$ respectivamente.

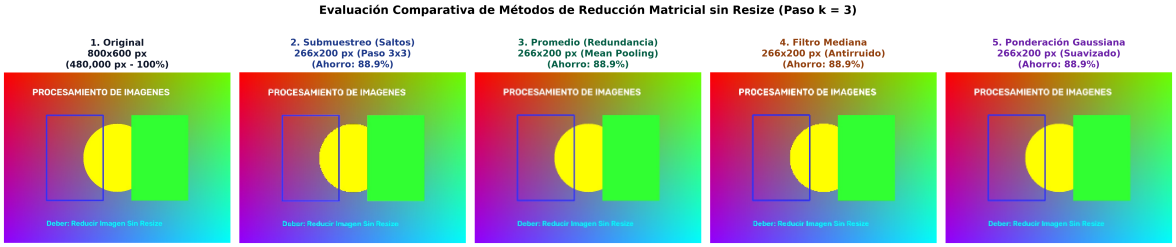


Figura 3: Comparativa visual de los cuatro métodos de reducción para paso impar $k = 3$ (reducción a 266×200 píxeles, 88,9 % de ahorro).

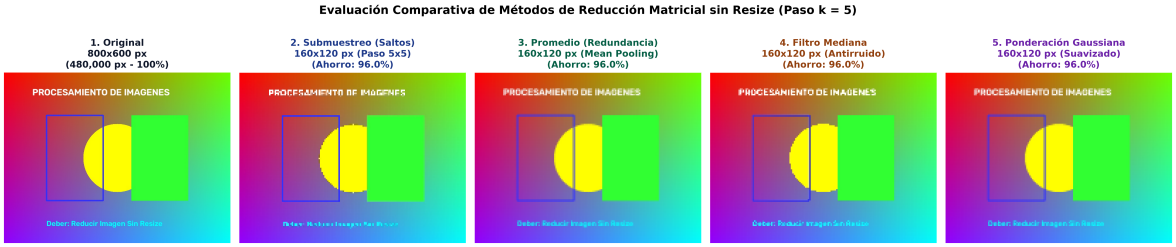


Figura 4: Comparativa visual de los cuatro métodos de reducción para paso impar $k = 5$ (reducción a 160×120 píxeles, 96,0 % de ahorro).

Del análisis detallado de las imágenes reducidas se desprenden las siguientes observaciones científicas:

1. **Conservación de la Inteligibilidad:** A pesar de reducir la cantidad de píxeles a tan solo un 11,1 % del original en $k = 3$ y un 4,0 % en $k = 5$, todos los métodos preservan plenamente las formas, contornos y contenido de la escena. Esto confirma de manera concluyente la hipótesis de la **redundancia espacial**: la vecindad inmediata contiene información altamente correlacionada que permite reconocer los objetos con una fracción mínima de muestras.

2. **Efecto de Aliasing en el Submuestreo por Saltos:** El método de saltos directos exhibe artefactos visibles de pixelado y bordes serrados en las líneas diagonales y texturas finas. Esto ocurre porque la decimación omite el filtrado de paso bajo previo, violando el criterio de Nyquist.

3. **Balance y Suavidad en Promedio y Gaussiana:** Tanto el método de promedio como la convolución gaussiana presentan transiciones tonales notablemente más suaves y naturales. La gaussiana, al concentrar su energía en el centro geométrico, evita la pérdida excesiva de contraste en bordes que a veces aqueja al promedio simple.

4. **Estabilidad del Filtro Mediana:** La mediana retiene la nitidez de los bordes estructurales con mayor vigor que el promedio, evitando la borrosidad difusa al tiempo que elimina cualquier ruido impulsivo localizado.

6. Banco de Preguntas Técnicas para la Sustentación

Para consolidar la preparación académica y defender los resultados obtenidos ante el docente evaluador, se estructuran las siguientes preguntas conceptuales y técnicas clave:

Pregunta 1: ¿Por qué el docente prohibió el uso de `imresize` o `cv2.resize`?
Respuesta: Porque dichas funciones aplican interpolaciones automáticas de caja negra (bilineal, bicúbica o Lanczos) que realizan aproximaciones polinomiales complejas, impidiendo evaluar el objetivo pedagógico del taller: comprender la manipulación directa de índices en matrices bidimensionales y el muestreo espacial discreto.

Pregunta 2: Si aplicamos un paso $k = 3$, ¿a cuánto se reduce la cantidad total de píxeles y por qué no a la

Respuesta: Se reduce a la **novena parte** ($\approx 11{,}11\,\%$ de los píxeles originales, es decir, un $88{,}89\,\%$ de ahorro), debido a que la reducción ocurre simultáneamente en dos dimensiones ortogonales: alto $(1/k)$ y ancho $(1/k)$, dando como factor de área:

$$\text{Factor de escala bidimensional} = \frac{1}{k} \times \frac{1}{k} = \frac{1}{k^2} = \frac{1}{3^2} = \frac{1}{9} \tag{15}$$

Pregunta 3: ¿Cuál es la diferencia matemática fundamental entre el método de saltos y el de promedio?

Respuesta: El método de saltos realiza una decimación espacial pura descartando $(k^2 - 1)$ píxeles del bloque sin procesarlos, lo que genera aliasing. El método de promedio calcula una combinación lineal uniforme ponderada por $1/k^2$, actuando como un filtro pasa-bajos que aprovecha la redundancia espacial y preserva la energía luminosa media de la escena.

Pregunta 4: ¿Por qué es matemáticamente obligatorio utilizar matrices de tamaño impar ($3 \times 3, 5 \times 5, 7 \times 7$

Respuesta: Por tres razones determinantes: 1) Tienen un centro geométrico entero simétrico en $((k-1)/2, (k-1)/2)$; 2) El cardinal k^2 es impar, lo que garantiza que la mediana sea un píxel real de la escena sin requerir promedios artificiales; y 3) Permiten construir máscaras gaussianas isotrópicas simétricas alrededor de su centro.

Pregunta 5: ¿Cómo se maneja una imagen a color frente a una en escala de grises?

Respuesta: En escala de grises se opera sobre una matriz bidimensional $M \times N$. En color se trabaja sobre un tensor tridimensional $M \times N \times 3$, aplicando el algoritmo de manera desacoplada e idéntica a cada canal cromático (Rojo, Verde y Azul) y recombinándolos posteriormente.

7. Conclusiones y Recomendaciones

7.1. Conclusiones

1. Se implementaron de forma exitosa los cuatro métodos de reducción dimensional matricial en Python (saltos, promedio, mediana y gaussiana) sin requerir funciones empaquetadas como `cv2.resize` o `imresize`, validando la eficacia del cálculo matricial directo mediante `NumPy`.

2. Se demostró cuantitativamente que el factor de reducción espacial escala de forma cuadrática $(1/k^2)$, logrando reducciones de datos del $88{,}92\,\%$ para $k = 3$, $96{,}00\,\%$ para $k = 5$ y $97{,}98\,\%$ para $k = 7$, lo que evidencia su utilidad crítica para optimizar el preprocesamiento de imágenes en sistemas de inteligencia artificial.

3. Se justificó analítica y geométricamente la necesidad de emplear bloques de orden impar ($3\times3, 5\times5, 7 \times 7$), dado que aseguran la existencia de un centro geométrico simétrico entero y posibilitan el cómputo de la mediana pura sin interpolaciones artificiales.

4. La comparación visual corroboró el principio de redundancia espacial: los píxeles adyacentes presentan una elevada correlación lumínica, lo cual permite que la imagen reducida conserve plenamente su interpretabilidad semántica y estructural.

7.2. Recomendaciones

1. Para sistemas de visión por computador en tiempo real (como detección de objetos en video con YOLO), se recomienda emplear el método de promedio o gaussiana con $k = 3$, ya que equilibra de manera óptima una reducción del $88{,}9\,\%$ de datos con una excelente atenuación del ruido y el aliasing.

2. En presencia de imágenes con sensores ruidosos o artefactos de transmisión de tipo sal y pimienta, se aconseja priorizar el método de mediana por su probada robustez no lineal.

3. Extender la presente práctica hacia la construcción de pirámides gaussianas y laplacianas multi-escala, base fundamental de algoritmos modernos de fusión de imágenes y detección de características como SIFT y SURF.

Referencias Bibliográficas

[1] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd. Cham, Switzerland: Springer, 2022.

[2] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, vol. 25, n.º 11, págs. 120-125, 2000.

[3] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4th. New York, NY, USA: Pearson, 2018.

[4] C. R. Harris, K. J. Millman, S. J. van der Walt et al., "Array programming with NumPy," *Nature*, vol. 585, n.º 7825, págs. 357-362, 2020.

[5] A. K. Jain, *Fundamentals of Digital Image Processing*. Englewood Cliffs, NJ, USA: Prentice Hall, 1989.

[6] C. E. Shannon, "Communication in the Presence of Noise," *Proceedings of the IRE*, vol. 37, n.º 1, págs. 10-21, 1949.

[7] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, n.º 3, págs. 90-95, 2007.